# UNIT-5

## Lecture-1

**Recommended system**

Recommender systems are used in machine learning to predict the ratings or preferences of items for a given user. They are commonly used in e-commerce applications to suggest items that a user may be interested in. One common example of a recommender system is Netflix. Netflix uses a recommender system to suggest movies and TV shows that a user may want to watch. The algorithm looks at past ratings and preferences to make suggestions. In this blog post, you will learn about recommender systems and some of the different types of recommender systems with the help of examples.

Recommender systems make use of machine learning to predict the ratings or preferences of items for a given user. They are commonly used in e-commerce applications to suggest items that a user may be interested in. Recommender systems are important for data scientists because they can be used to make suggestions about what a person may want to buy or watch.

**What is a Recommender System?**

A **recommender system,** or a recommendation system, is a subclass of **information filtering system** that seeks to predict the "rating" or "preference" a user would give to an item. Recommender systems are utilized in a variety of areas, with commonly recognized examples taking the form of playlist generators for video and music services, product recommenders for online stores, etc. These systems can operate using a single input, like music, or multiple inputs within and across platforms like news, books, and search queries. Recommender systems are utilized in order to make better product suggestions to customers, or personalized recommendations to friends.

Recommender systems leverage **machine learning algorithms** in order to make better **predictions** about a **user's preferences**. There are a number of different machine learning algorithms that can be used in a recommender system. Each algorithm has its own strengths and weaknesses, and the best algorithm for a particular application will depend on the nature of the data. The most common is the linear regression algorithm. The linear regression algorithm is used to find the best linear approximation to a data set. In a recommender system, this algorithm is used to predict how a user will rate an item based on their past ratings. Other machine learning algorithms that can be used in Recommender Systems, include some of the following:

- **K-NearestNeighbor (K-NN)**: The key idea behind using K-NN in recommender systems is to find the **nearest neighbors (similar users or items)** based on historical data and use their preferences or interactions to make recommendations. In **user-based collaborative filtering**, K-NN can be used to find the k most similar users to a target user based on their historical interactions with items. The similarity can be measured using various distance metrics, such as cosine similarity or Pearson correlation coefficient. Once the similar users are identified, the system can recommend items that these users have liked or interacted

with. In **item-based collaborative filtering**, K-NN can be used to find the k most similar items to a target item based on user preferences. The similarity can be calculated using techniques like cosine similarity or adjusted cosine similarity. Once similar items are identified, the system can recommend these items to users who have shown interest in the target item. It's worth noting that **K-NN has some limitations**, such as scalability issues with large datasets and the sparsity problem (when there are few overlapping interactions between users or items).

- **Bayesian inference**: Bayesian inference can be used in building recommender systems, particularly in the context of probabilistic graphical models. Bayesian inference allows for the incorporation of prior knowledge and uncertainty into the recommendation process. One approach that utilizes Bayesian inference is **Bayesian Personalized Ranking (BPR)**. BPR is a popular method for collaborative filtering in recommender systems. It is based on pairwise ranking and uses Bayesian principles to model the preferences of users. In BPR, a probabilistic model is constructed to estimate the likelihood of a user's preference for one item over another. The model incorporates prior knowledge about user-item interactions and learns the parameters that maximize the likelihood of observed rankings. The Bayesian framework enables the system to update its beliefs about user preferences as more data is observed.
- **Dimensionality reduction**: Dimensionality reduction techniques are commonly used in building recommender systems to address the "curse of dimensionality" and improve the efficiency and performance of the recommendation process. These techniques aim to reduce the number of dimensions or features in the data while preserving important information or structure. There are two main ways dimensionality reduction is used in recommender systems.
    - **Matrix factorization**: Matrix factorization is a dimensionality reduction technique widely used in collaborative filtering-based recommender systems. It aims to decompose the user-item interaction matrix into lower-dimensional representations or latent factors. The idea is to represent users and items in a shared latent space where their preferences and characteristics are captured. This reduces the dimensionality of the original data and allows for efficient computation of recommendations.
    - **Feature extraction**: In content-based filtering approaches, dimensionality reduction techniques can be used to extract meaningful features from item attributes or content. For example, in text-based recommender systems, techniques like Latent Semantic Analysis (LSA) or Latent Dirichlet Allocation (LDA) can be used to reduce the dimensionality of the text data and extract latent topics or features.
- **Neural Networks**: A neural network is a type of machine learning algorithm that is similar to the brain. It is composed of interconnected neurons that can learn to recognize patterns. Neural networks are often used for prediction tasks, like recommender systems.

The following is a list of benefits / value of building recommender system and why businesses must consider:

- Recommender systems help businesses make money by increasing sales and predicting what customers want.
- Recommender systems can improve customer satisfaction by providing relevant recommendations. They can improve customer loyalty by suggesting new items that the customer may like.

- Recommender systems can reduce the amount of time it takes to find the right item for a customer.
- Recommender systems can help businesses learn more about their customers' preferences.

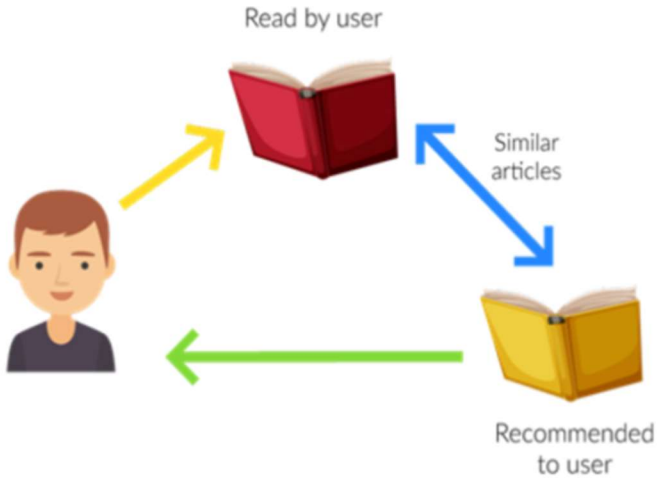
# Lecture-2

**Types of Recommender Systems**

There are a number of different types of recommender systems. Some of them are listed below:

- Content-based recommendation
- Collaborative filtering based recommendation
- Hybrid recommender system

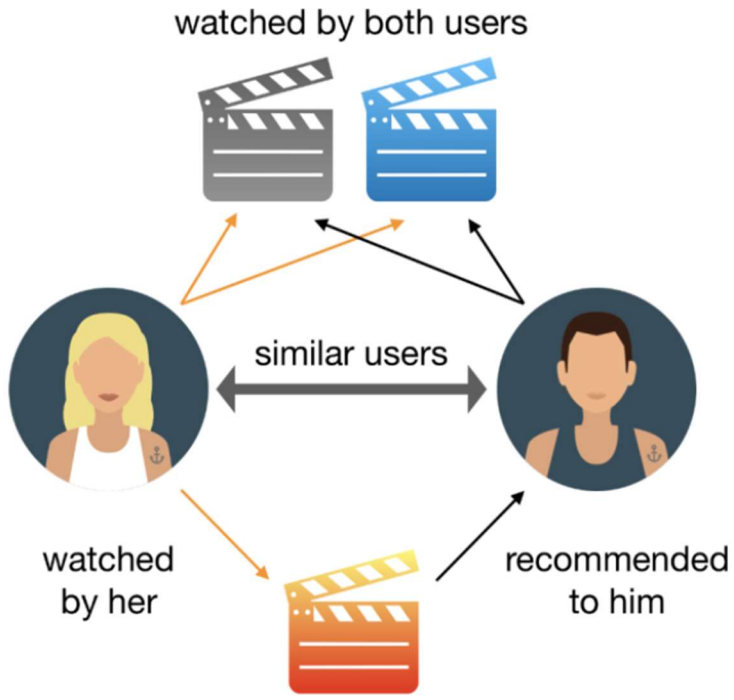
**Content based recommender system**
The most common type of recommender system is the **content-based recommender system**. A content-based recommender system is a type of recommender system that relies on the similarity between items to make recommendations. For example, if you're looking for a new movie to watch, a content-based recommender system might recommend movies that are similar to ones you've watched in the past. The picture below represents content-based filtering recommender system:




Content-based recommender systems are commonly used in music, books, and movies. They can be used to recommend products, services, or even websites. Content-based recommender systems are based on the idea that if you like one item, you're likely to like other items that are similar to it. To build a content-based recommender system, you need to first define what similarity means. This is where machine learning comes in. A content-based recommender system can use machine learning to learn the similarity between items. Once it has learned the similarity between items, it can make recommendations accordingly.

**Collaborative filtering recommender system**
Another type of recommender system is the **collaborative filtering recommender system**. A collaborative filtering recommender system is a type of machine learning algorithm that makes

predictions about what a user might want to buy or watch based on the past behavior of other users. The algorithm looks at the items that other users with similar taste have purchased or rated highly, and recommends those items to the new user. The picture below represents collaborative-filtering recommender system:




The main advantage of collaborative filtering is that it doesn't require any information about the users or items; all it needs is a dataset of past user behavior. Collaborative filtering is one of the most popular techniques for building recommender systems, and is used by major companies such as Amazon, Netflix, and Spotify.

**Hybrid recommender system**

A third type of recommender system is the **hybrid recommender system**. The hybrid approach has become increasingly popular in recent years as it offers the potential to overcome some of the limitations of each individual approach.

Content-based recommender systems focus on the attributes of items in order to make recommendations. This can often lead to issues with scalability, as the system needs to be constantly updated with new content in order to make accurate recommendations. Collaborative filtering systems, on the other hand, focus on the relationships between users and items. This can often result in problems with sparsity, as it can be difficult to find enough users who have rated a given item. The hybrid approach seeks to overcome these limitations by combining the two approaches.

A hybrid recommender system is a type of recommender system that combines both content-based and collaborative filtering approaches. The hybrid approach takes advantage of both

content-based and collaborative filtering by using them to supplement each other. For example, a hybrid recommender system might first identify a set of items that are similar to the item the user is interested in, and then use collaborative filtering to identify which of those items the user is most likely to enjoy. This approach can provide more accurate recommendations than either method used alone. The hybrid approach has been shown to be more effective than either method used alone, as it is able to leverage the strengths of both approaches. Hybrid recommender systems are often more scalable and efficient than pure content-based or collaborative filtering systems.

**Examples of recommender systems**

Some of the most popular examples of recommender systems include the ones used by Amazon, Netflix, and Spotify.

- Amazon's recommender system is based on a combination of collaborative filtering and content-based algorithms. It uses past customer behavior to make recommendations for new products. Amazon's recommender system is one of the most complex and sophisticated in the world.
- Netflix's recommender system is also based on a combination of collaborative filtering and content-based algorithms. However, Netflix takes things a step further by also incorporating machine learning into its algorithm. This allows Netflix to make predictions about what a user might want to watch based on the behavior of other users.
- Spotify's recommender system is based on collaborative filtering. It uses past user behavior to make recommendations for new songs to listen to.

# Lecture-3

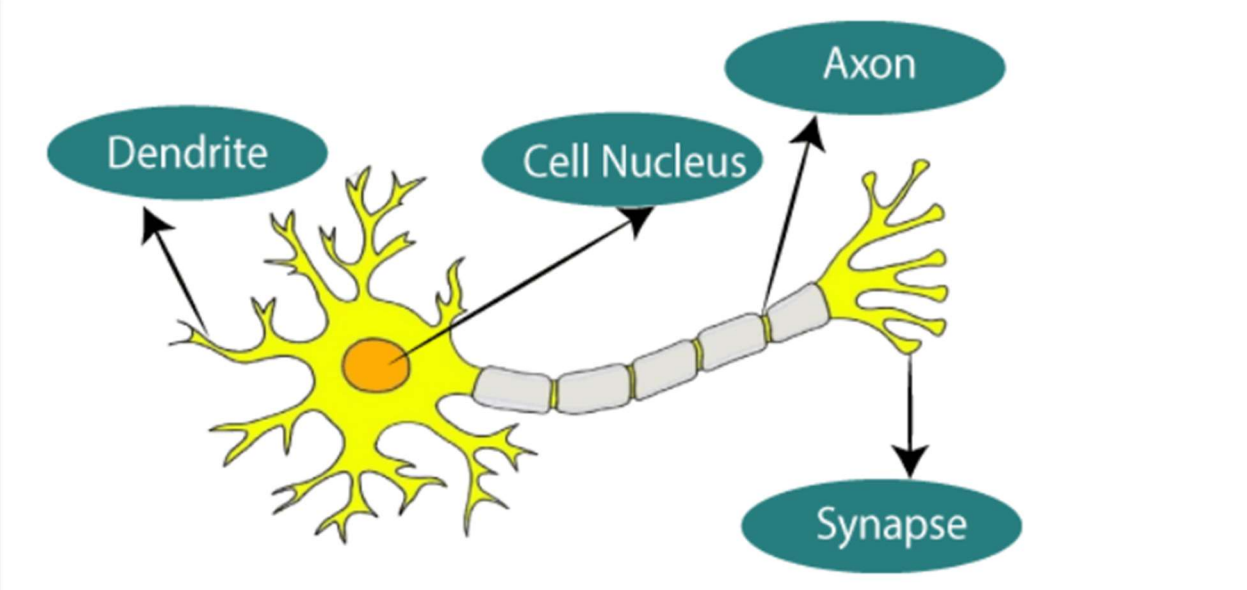
**Artificial neural network**

The term "Artificial neural network" refers to a biologically inspired sub-field of artificial intelligence modeled after the brain. An Artificial neural network is usually a computational network based on biological neural networks that construct the structure of the human brain. Similar to a human brain has neurons interconnected to each other, artificial neural networks also have neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

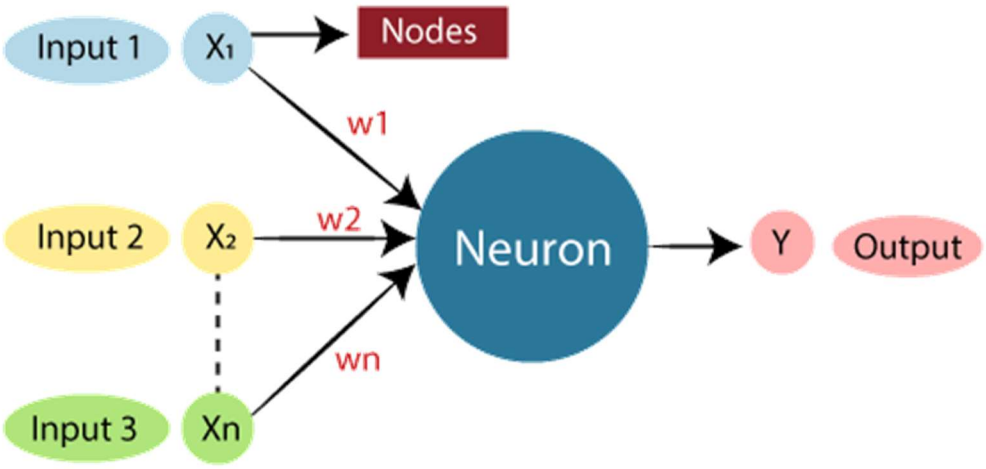
Artificial neural network tutorial covers all the aspects related to the artificial neural network. In this tutorial, we will discuss ANNs, Adaptive resonance theory, Kohonen self-organizing map, Building blocks, unsupervised learning, Genetic algorithm, etc.

**What is Artificial Neural Network?**

The term "**Artificial Neural Network**" is derived from Biological neural networks that develop the structure of a human brain. Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.




**The typical Artificial Neural Network looks something like the given figure.**




Dendrites from Biological Neural Network represent inputs in Artificial Neural Networks, cell nucleus represents Nodes, synapse represents Weights, and Axon represents Output.

Relationship between Biological neural network and artificial neural network:

| **Biological Neural Network** | **Artificial Neural Network** |
|---|---|
| Dendrites | Inputs |
| Cell nucleus | Nodes |
| Synapse | Weights |
| Axon | Output |

An **Artificial Neural Network** in the field of **Artificial intelligence** where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner. The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.

There are around 1000 billion neurons in the human brain. Each neuron has an association point somewhere in the range of 1,000 and 100,000. In the human brain, data is stored in such a manner as to be distributed, and we can extract more than one piece of this data when necessary from our memory parallelly. We can say that the human brain is made up of incredibly amazing parallel processors.

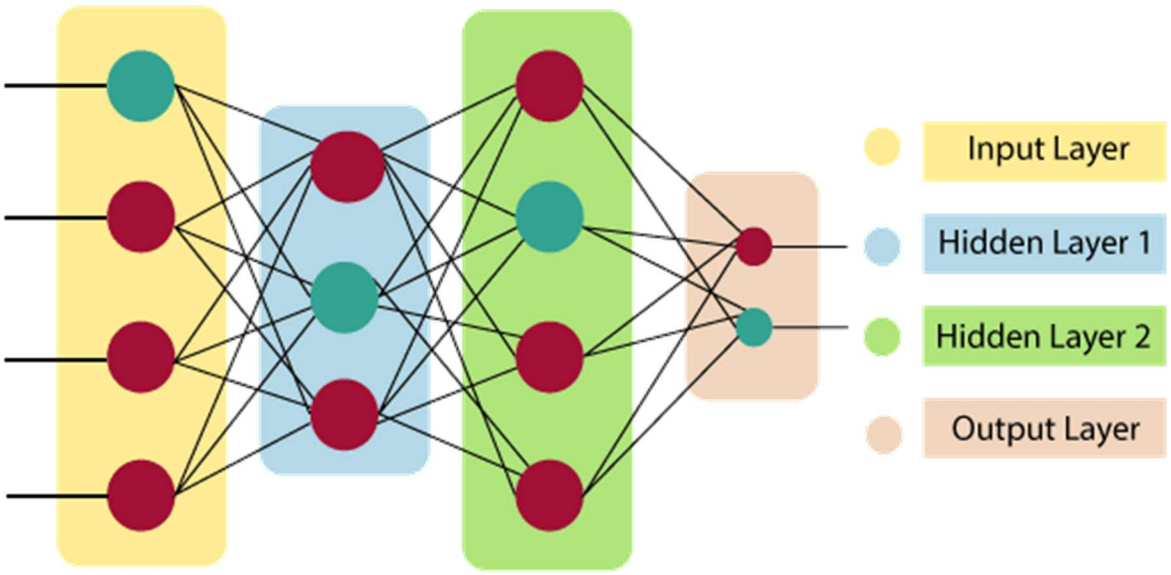
We can understand the artificial neural network with an example, consider an example of a digital logic gate that takes an input and gives an output. "OR" gate, which takes two inputs. If one or both the inputs are "On," then we get "On" in output. If both the inputs are "Off," then we get "Off" in output. Here the output depends upon input. Our brain does not perform the same

task. The outputs to inputs relationship keep changing because of the neurons in our brain, which are "learning."

**Architecture of an artificial neural network**

To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers. Lets us look at various types of layers available in an artificial neural network.

Artificial Neural Network primarily consists of three layers:




**Input Layer:**

As the name suggests, it accepts inputs in several different formats provided by the programmer.

**Hidden Layer:**

The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

**Output Layer:**

The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^{n} Wi * Xi + b$$

It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.

**Advantages of Artificial Neural Network (ANN)**

**Parallel processing capability:**

Artificial neural networks have a numerical value that can perform more than one task simultaneously.

**Storing data on the entire network:**

Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.

**Capability to work with incomplete knowledge:**

After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.

**Having a memory distribution:**

For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network. The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.

**Having fault tolerance:**

Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

**Disadvantages of Artificial Neural Network:**

**Assurance of proper network structure:**

There is no particular guideline for determining the structure of artificial neural networks. The appropriate network structure is accomplished through experience, trial, and error.

**Unrecognized behavior of the network:**

It is the most significant issue of ANN. When ANN produces a testing solution, it does not provide insight concerning why and how. It decreases trust in the network.

**Hardware dependence:**

Artificial neural networks need processors with parallel processing power, as per their structure. Therefore, the realization of the equipment is dependent.

**Difficulty of showing the issue to the network:**

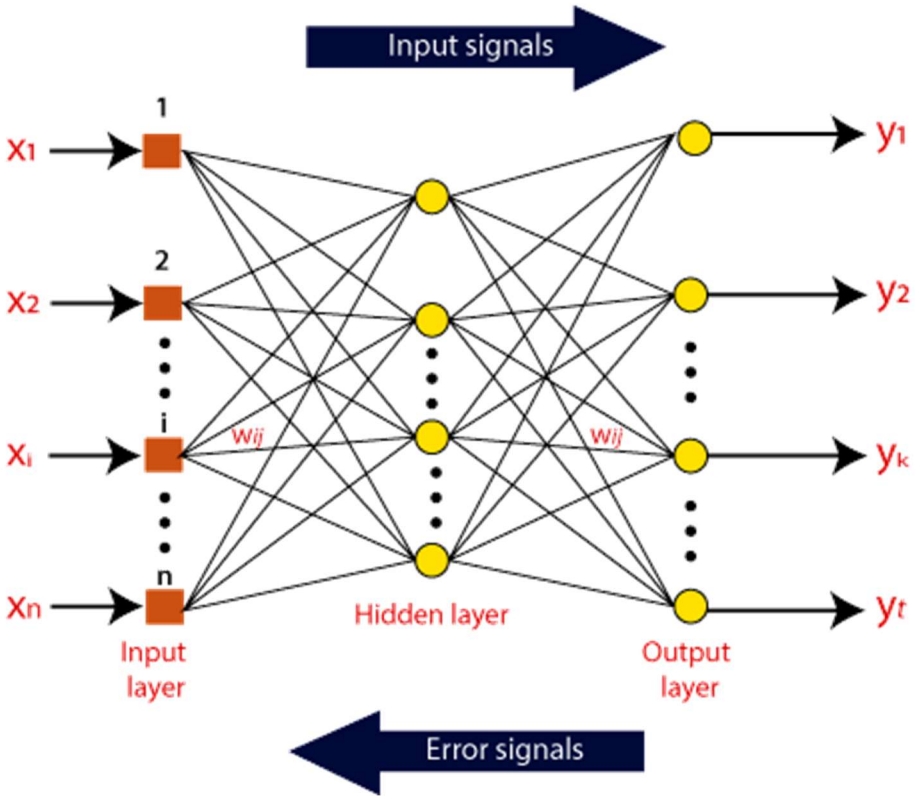
ANNs can work with numerical data. Problems must be converted into numerical values before being introduced to ANN. The presentation mechanism to be resolved here will directly impact the performance of the network. It relies on the user's abilities.

**The duration of the network is unknown:**

The network is reduced to a specific value of the error, and this value does not give us optimum results.

**How do artificial neural networks work?**

Artificial Neural Network can be best represented as a weighted directed graph, where the artificial neurons form the nodes. The association between the neurons outputs and neuron inputs can be viewed as the directed edges with weights. The Artificial Neural Network receives the input signal from the external source in the form of a pattern and image in the form of a vector. These inputs are then mathematically assigned by the notations x(n) for every n number of inputs.




Afterward, each of the input is multiplied by its corresponding weights ( these weights are the details utilized by the artificial neural networks to solve a specific problem ). In general terms, these weights normally represent the strength of the interconnection between neurons inside the artificial neural network. All the weighted inputs are summarized inside the computing unit.

If the weighted sum is equal to zero, then bias is added to make the output non-zero or something else to scale up to the system's response. Bias has the same input, and weight equals to 1. Here the total of weighted inputs can be in the range of 0 to positive infinity. Here, to keep the response in the limits of the desired value, a certain maximum value is benchmarked, and the total of weighted inputs is passed through the activation function.

The activation function refers to the set of transfer functions used to achieve the desired output. There is a different kind of the activation function, but primarily either linear or non-linear sets of functions. Some of the commonly used sets of activation functions are the Binary, linear, and Tan hyperbolic sigmoidal activation functions. Let us take a look at each of them in details:

### Binary:

In binary activation function, the output is either a one or a 0. Here, to accomplish this, there is a threshold value set up. If the net weighted input of neurons is more than 1, then the final output of the activation function is returned as one or else the output is returned as 0.

### Sigmoidal Hyperbolic:

The Sigmoidal Hyperbola function is generally seen as an "**S**" shaped curve. Here the tan hyperbolic function is used to approximate output from the actual net input. The function is defined as:

**F(x) = (1/1 + exp(-????x))**

Where ???? is considered the Steepness parameter.

# Lecture-4

## Perceptron

What is the Perceptron model in Machine Learning?

Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. Further, ***Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence***.

Perceptron model is also treated as one of the best and simplest types of Artificial Neural networks. However, it is a supervised learning algorithm of binary classifiers. Hence, we can consider it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**

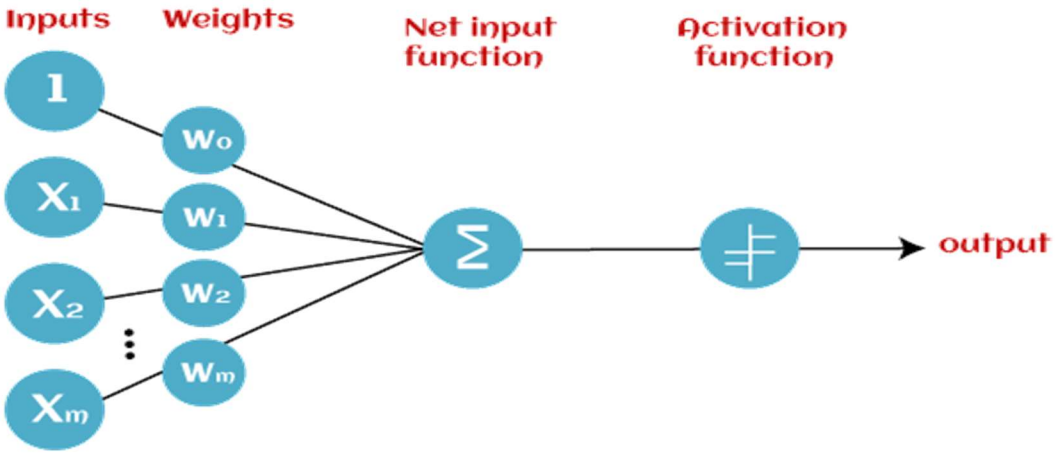
What is Binary classifier in Machine Learning?

In Machine Learning, binary classifiers are defined as the function that helps in deciding whether input data can be represented as vectors of numbers and belongs to some specific class.

Binary classifiers can be considered as linear classifiers. In simple words, we can understand it as a ***classification algorithm that can predict linear predictor function in terms of weight and feature vectors.***

## Basic Components of Perceptron

Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:




- **Input Nodes or Input Layer:**

This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

- **Wight and Bias:**

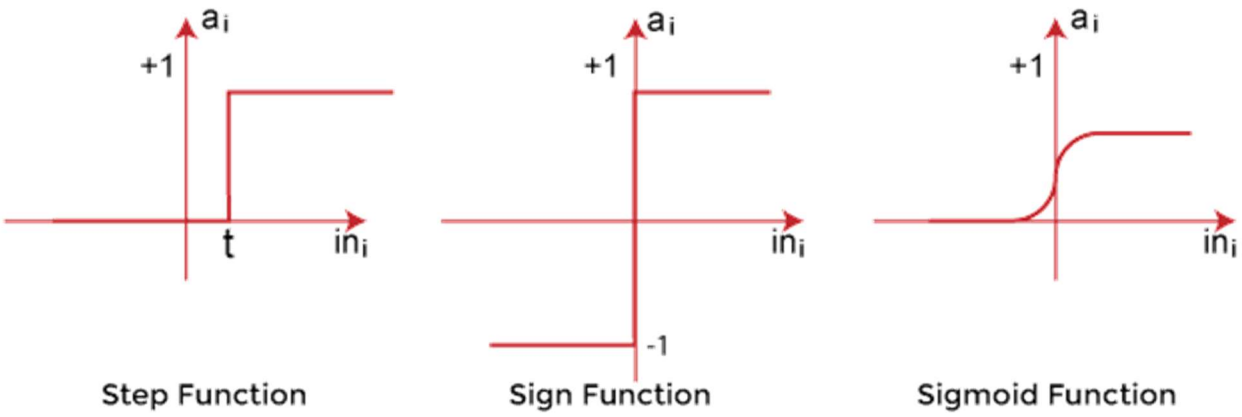
Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

- **Activation Function:**

These are the final and important components that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function.

Types of Activation functions:

- Sign function
- Step function, and
- Sigmoid function

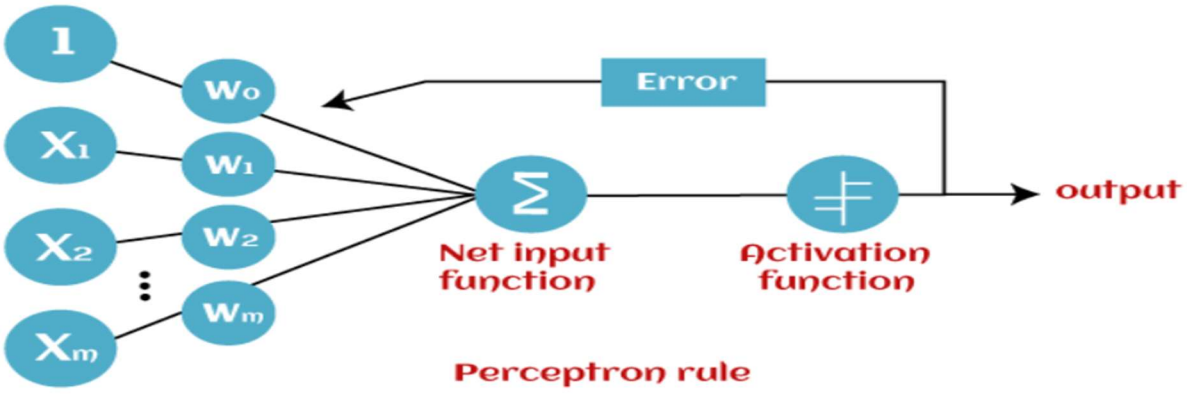



The data scientist uses the activation function to take a subjective decision based on various problem statements and forms the desired outputs. Activation function may differ (e.g., Sign, Step, and Sigmoid) in perceptron models by checking whether the learning process is slow or has vanishing or exploding gradients.

**How does Perceptron work?**

In Machine Learning, Perceptron is considered as a single-layer neural network that consists of four main parameters named input values (Input nodes), weights and Bias, net sum, and an activation function. The perceptron model begins with the multiplication of all input values and

their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the **step function** and is represented by **'f'**.




This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1). It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.

Perceptron model works in two important steps as follows:

**Step-1**

In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

$\sum wi*xi = x1*w1 + x2*w2 + \ldots wn*xn$

Add a special term called **bias 'b'** to this weighted sum to improve the model's performance.

**$\sum wi*xi + b$**

**Step-2**

In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:

**$Y = f(\sum wi*xi + b)$**

## Characteristics of Perceptron

The perceptron model has the following characteristics.

1. Perceptron is a machine learning algorithm for supervised learning of binary classifiers.
2. In Perceptron, the weight coefficient is automatically learned.
3. Initially, weights are multiplied with input features, and the decision is made whether the neuron is fired or not.
4. The activation function applies a step rule to check whether the weight function is greater than zero.
5. The linear decision boundary is drawn, enabling the distinction between the two linearly separable classes +1 and -1.
6. If the added sum of all input values is more than the threshold value, it must have an output signal; otherwise, no output will be shown.

## Limitations of Perceptron Model

**A perceptron model has limitations as follows:**

- The output of a perceptron can only be a binary number (0 or 1) due to the hard limit transfer function.
- Perceptron can only be used to classify the linearly separable sets of input vectors. If input vectors are non-linear, it is not easy to classify them properly.

# Lecture-5

## Multilayer network

A multilayer artificial neuron network is an integral part of deep learning. And this lesson will help you with an overview of multilayer ANN along with overfitting and underfitting. Not just that, by the end of the lesson you will also learn:

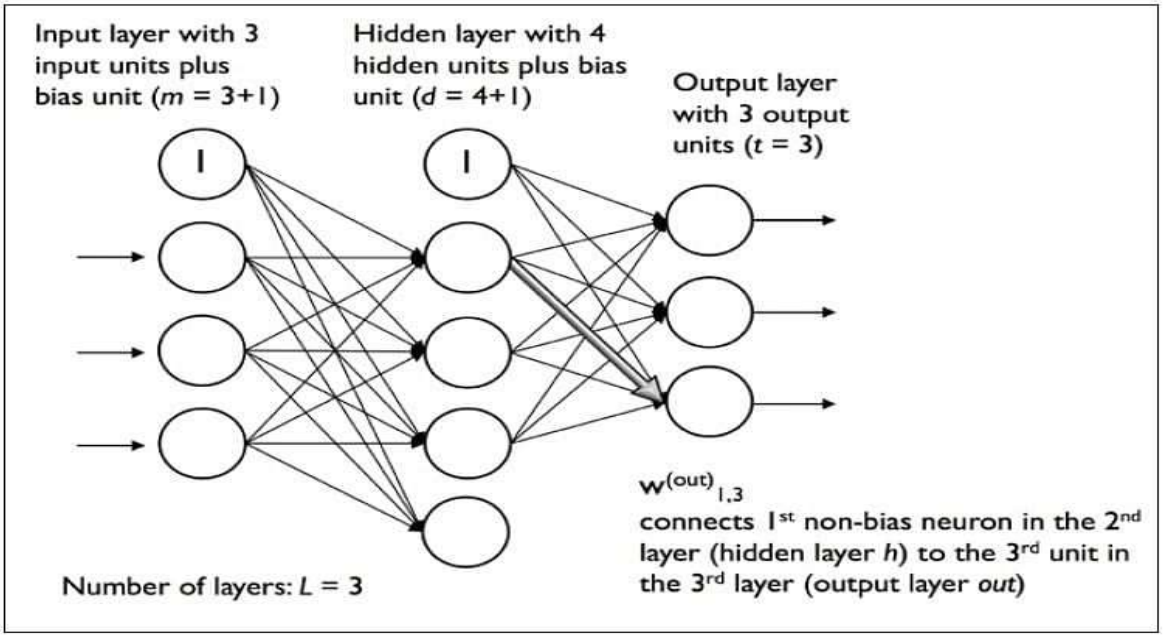
- Analyze how to regularize and minimize the cost function in a neural network
- Carry out backpropagation to adjust weights in a neural network
- Inspect convergence in a multilayer ANN
- Explore multilayer ANN
- Implement forward propagation in multilayer perceptron (MLP)
- Understand how the capacity of a model is affected by underfitting and overfitting

## History of Multi-layer ANN

Deep Learning deals with training multi-layer artificial neural networks, also called Deep Neural Networks. After Rosenblatt perceptron was developed in the 1950s, there was a lack of interest in neural networks until 1986, when Dr.Hinton and his colleagues developed the backpropagation algorithm to train a multilayer neural network. Today it is a hot topic with many leading firms like Google, Facebook, and Microsoft which invest heavily in applications using deep neural networks.

## Multi-layer ANN

A fully connected multi-layer neural network is called a Multilayer Perceptron (MLP).




It has 3 layers including one hidden layer. If it has more than 1 hidden layer, it is called a deep ANN. An MLP is a typical example of a feedforward artificial neural network. In this figure, the ith activation unit in the lth layer is denoted as ai(l).

The number of layers and the number of neurons are referred to as hyperparameters of a neural network, and these need tuning. Cross-validation techniques must be used to find ideal values for these.

The weight adjustment training is done via backpropagation. Deeper neural networks are better at processing data. However, deeper layers can lead to vanishing gradient problems. Special algorithms are required to solve this issue.

**Notations**

In the representation below:

$$a^{(in)} = \begin{bmatrix} a_0^{(in)} \\ a_1^{(in)} \\ \vdots \\ a_m^{(in)} \end{bmatrix} = \begin{bmatrix} 1 \\ x_1^{(in)} \\ \vdots \\ x_m^{(in)} \end{bmatrix}$$

- ai(in) refers to the ith value in the input layer
- ai(h) refers to the ith unit in the hidden layer
- ai(out) refers to the ith unit in the output layer
- ao(in) is simply the bias unit and is equal to 1; it will have the corresponding weight w0
- The weight coefficient from layer l to layer l+1 is represented by wk,j(l)

A simplified view of the multilayer is presented here. This image shows a fully connected three-layer neural network with 3 input neurons and 3 output neurons. A bias term is added to the input vector.

**Forward Propagation**

In the following topics, let us look at the forward propagation in detail.

The MLP learning procedure is as follows:

- Starting with the input layer, propagate data forward to the output layer. This step is the forward propagation.
- Based on the output, calculate the error (the difference between the predicted and known outcome). The error needs to be minimized.
- Backpropagate the error. Find its derivative with respect to each weight in the network, and update the model.

Repeat the three steps given above over multiple epochs to learn ideal weights.

Finally, the output is taken via a threshold function to obtain the predicted class labels.

**Forward Propagation in MLP**

In the first step, calculate the activation unit al(h) of the hidden layer.

$$z_1^{(h)} = a_0^{(in)} w_{0,1}^{(h)} + a_1^{(in)} w_{1.1}^{(h)} + \cdots + a_m^{(in)} w_{m,1}^{(h)}$$

$$a_1^{(h)} = \phi\left( z_1^{(h)} \right)$$

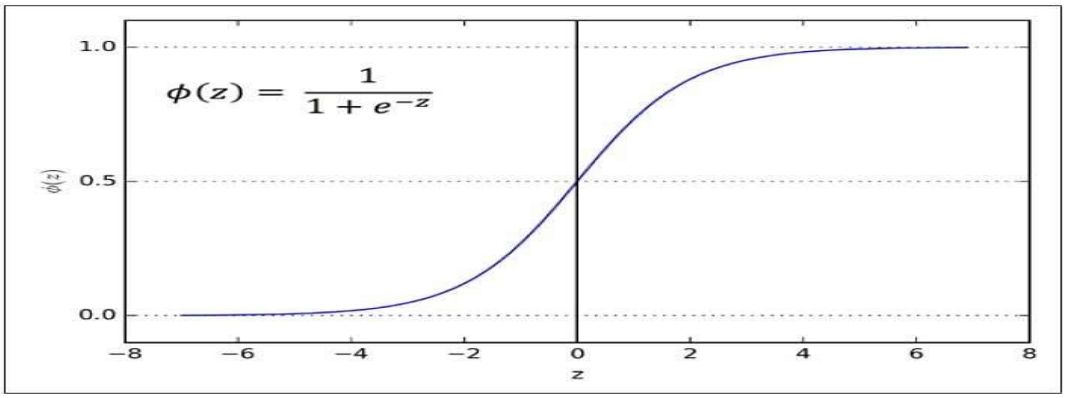
Activation unit is the result of applying an activation function φ to the z value. It must be differentiable to be able to learn weights using gradient descent. The activation function φ is often the sigmoid (logistic) function.

$$\phi(z) = \frac{1}{1+e^{-z}}$$

It allows nonlinearity needed to solve complex problems like image processing.

**Sigmoid Curve**

The sigmoid curve is an S-shaped curve.



# Lecture-6

**Back propagation**

Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

**Features of Backpropagation:**

1. it is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.
2. it is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
3. training is done in the three stages :
   - the feed-forward of input training pattern
   - the calculation and backpropagation of the error
   - updation of the weight.

**Working of Backpropagation:**

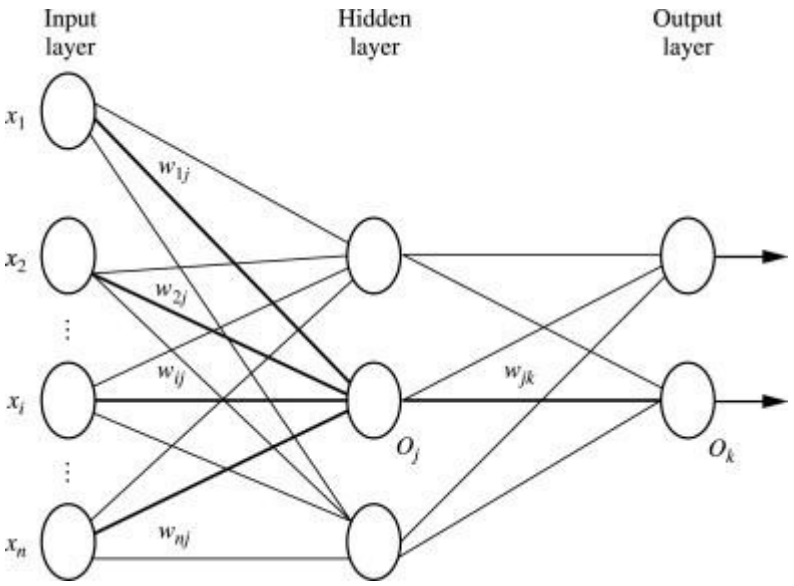
Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

**Backpropagation Algorithm:**

**Step 1:** Inputs X, arrive through the preconnected path.
**Step 2:** The input is modeled using true weights W. Weights are usually chosen randomly.
**Step 3:** Calculate the output of each neuron from the input layer to the hidden layer to the output layer.
**Step 4:** Calculate the error in the outputs
Backpropagation Error= Actual Output – Desired Output

**Step 5:** From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

**Step 6:** Repeat the process until the desired output is achieved.




**Parameters :**

- x = inputs training vector $x=(x_1,x_2,\ldots\ldots\ldots\ldots x_n)$.
- t = target vector $t=(t_1,t_2\ldots\ldots\ldots\ldots t_{n)}$.
- $\delta_k$ = error at output unit.
- $\delta_j$ = error at hidden layer.
- $\alpha$ = learning rate.
- $v_{0j}$ = bias of hidden unit j.

**Training Algorithm :**

**Step 1:** Initialize weight to small random values.

**Step 2:** While the steps stopping condition is to be false do step 3 to 10.

**Step 3:** For each training pair do step 4 to 9 (Feed-Forward).

**Step 4:** Each input unit receives the signal unit and transmits the signal $x_i$ signal to all the units.

**Step 5 :** Each hidden unit Zj (z=1 to a) sums its weighted input signal to calculate its net input

$$z_{inj} = v_{0j} + \Sigma x_i v_{ij} \quad (i=1 \text{ to } n)$$

Applying activation function $z_j = f(z_{inj})$ and sends this signals to all units in the layer about i.e output units

For each output l=unit $y_k$ = (k=1 to m) sums its weighted input signals.

$$y_{ink} = w_{0k} + \Sigma z_i w_{jk} \quad (j=1 \text{ to } a)$$

and applies its activation function to calculate the output signals.

$$y_k = f(y_{ink})$$

**Backpropagation Error :**

**Step 6:** Each output unit $y_k$ (k=1 to n) receives a target pattern corresponding to an input pattern then error is calculated as:

$$\delta_k = ( t_k - y_k ) + y_{ink}$$

**Step 7:** Each hidden unit $Z_j$ (j=1 to a) sums its input from all units in the layer above

$$\delta_{inj} = \Sigma \delta_j w_{jk}$$

The error information term is calculated as :

$$\delta_j = \delta_{inj} + z_{inj}$$

**Updation of weight and bias :**

**Step 8:** Each output unit $y_k$ (k=1 to m) updates its bias and weight (j=1 to a). The weight correction term is given by :

$$\Delta w_{jk} = \alpha \delta_k z_j$$

and the bias correction term is given by $\Delta w_k = \alpha \delta_k$.

therefore $w_{jk(new)} = w_{jk(old)} + \Delta w_{jk}$

$$w_{0k(new)} = w_{ok(old)} + \Delta w_{ok}$$

for each hidden unit $z_j$ (j=1 to a) update its bias and weights (i=0 to n) the weight connection term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

and the bias connection on term

$$\Delta v_{0j} = \alpha \delta_j$$

Therefore $v_{ij(new)} = v_{ij(old)} + \Delta v_{ij}$

$$v_{0j(new)} = v_{0j(old)} + \Delta v_{0j}$$

**Step 9:** Test the stopping condition. The stopping condition can be the minimization of error, number of epochs.

**Need for Backpropagation:**

Backpropagation is “backpropagation of errors” and is very useful for training neural networks. It’s fast, easy to implement, and simple. Backpropagation does not require any parameters to be set, except the number of inputs. Backpropagation is a flexible method because no prior knowledge of the network is required.

**Types of Backpropagation**

There are two types of backpropagation networks.

- **Static backpropagation:** Static backpropagation is a network designed to map static inputs for static outputs. These types of networks are capable of solving static classification problems such as OCR (Optical Character Recognition).
- **Recurrent backpropagation:** Recursive backpropagation is another network used for fixed-point learning. Activation in recurrent backpropagation is feed-forward until a fixed value is reached. Static backpropagation provides an instant mapping, while recurrent backpropagation does not provide an instant mapping.

**Advantages:**

- It is simple, fast, and easy to program.
- Only numbers of the input are tuned, not any other parameter.
- It is Flexible and efficient.
- No need for users to learn any special functions.

**Disadvantages:**

- It is sensitive to noisy data and irregularities. Noisy data can lead to inaccurate results.
- Performance is highly dependent on input data.
- Spending too much time training.
- The matrix-based approach is preferred over a mini-batch.

# Lecture-7

**Introduction to Deep learning.**

Deep learning is a machine learning technique that teaches computers to do what comes naturally to humans: learn by example. Deep learning is a key technology behind driverless cars, enabling them to recognize a stop sign, or to distinguish a pedestrian from a lamppost. It is the key to voice control in consumer devices like phones, tablets, TVs, and hands-free speakers. Deep learning is getting lots of attention lately and for good reason. It's achieving results that were not possible before.

In deep learning, a computer model learns to perform classification tasks directly from images, text, or sound. Deep learning models can achieve state-of-the-art accuracy, sometimes exceeding human-level performance. Models are trained by using a large set of labeled data and neural network architectures that contain many layers.

**How does deep learning attain such impressive results?**

In a word, accuracy. Deep learning achieves recognition accuracy at higher levels than ever before. This helps consumer electronics meet user expectations, and it is crucial for safety-critical applications like driverless cars. Recent advances in deep learning have improved to the point where deep learning outperforms humans in some tasks like classifying objects in images.

While deep learning was first theorized in the 1980s, there are two main reasons it has only recently become useful:

1. Deep learning requires large amounts of **labeled data**. For example, driverless car development requires millions of images and thousands of hours of video.
2. Deep learning requires substantial **computing power**. High-performance GPUs have a parallel architecture that is efficient for deep learning. When combined with clusters or cloud computing, this enables development teams to reduce training time for a deep learning network from weeks to hours or less.

**Examples of Deep Learning at Work**

Deep learning applications are used in industries from automated driving to medical devices.

Automated Driving: Automotive researchers are using deep learning to automatically detect objects such as stop signs and traffic lights. In addition, deep learning is used to detect pedestrians, which helps decrease accidents.

Aerospace and Defense: Deep learning is used to identify objects from satellites that locate areas of interest, and identify safe or unsafe zones for troops.

Medical Research: Cancer researchers are using deep learning to automatically detect cancer cells. Teams at UCLA built an advanced microscope that yields a high-dimensional data set used to train a deep learning application to accurately identify cancer cells.

Industrial Automation: Deep learning is helping to improve worker safety around heavy machinery by automatically detecting when people or objects are within an unsafe distance of machines.

Electronics: Deep learning is being used in automated hearing and speech translation. For example, home assistance devices that respond to your voice and know your preferences are powered by deep learning applications.

**How Deep Learning Works**

Most deep learning methods use **neural network** architectures, which is why deep learning models are often referred to as **deep neural networks**.

The term "deep" usually refers to the number of hidden layers in the neural network. Traditional neural networks only contain 2-3 hidden layers, while deep networks can have as many as 150.

Deep learning models are trained by using large sets of labeled data and neural network architectures that learn features directly from the data without the need for manual feature extraction.

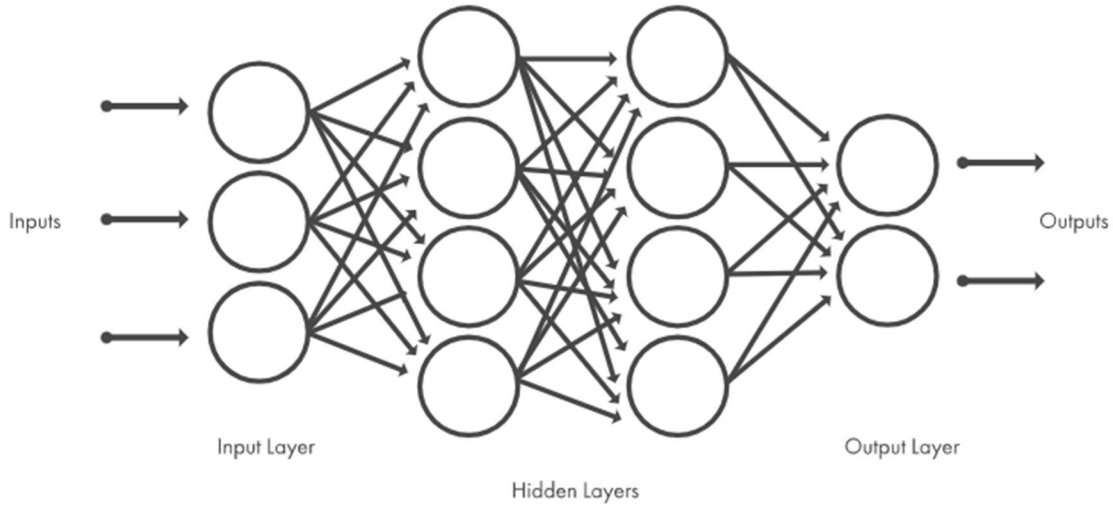



Figure 1: Neural networks, which are organized in layers consisting of a set of interconnected nodes. Networks can have tens or hundreds of hidden layers.

One of the most popular types of deep neural networks is known as convolutional neural networks **(CNN** or **ConvNet)**. A CNN convolves learned features with input data, and uses 2D convolutional layers, making this architecture well suited to processing 2D data, such as images.

CNNs eliminate the need for manual feature extraction, so you do not need to identify features used to classify images. The CNN works by extracting features directly from images. The relevant features are not pretrained; they are learned while the network trains on a collection of images. This automated feature extraction makes deep learning models highly accurate for computer vision tasks such as object classification.

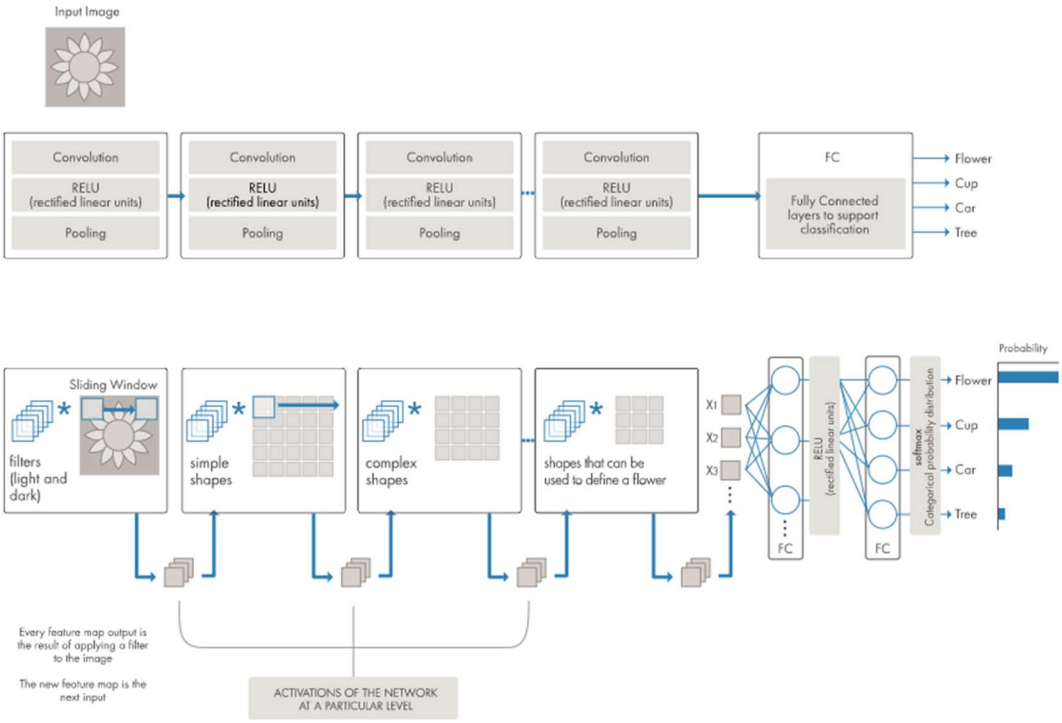



Figure 2: Example of a network with many convolutional layers. Filters are applied to each training image at different resolutions, and the output of each convolved image serves as the input to the next layer.

CNNs learn to detect different features of an image using tens or hundreds of hidden layers. Every hidden layer increases the complexity of the learned image features. For example, the first hidden layer could learn how to detect edges, and the last learns how to detect more complex shapes specifically catered to the shape of the object we are trying to recognize.

**Difference between Machine Learning and Deep Learning :**

Machine learning and deep learning both are subsets of artificial intelligence but there are many similarities and differences between them.

| Machine Learning | Deep Learning |
|---|---|
| Apply statistical algorithms to learn the | Uses artificial neural network architecture |

| Machine Learning | Deep Learning |
| --- | --- |
| hidden patterns and relationships in the dataset. | to learn the hidden patterns and relationships in the dataset. |
| Can work on the smaller amount of dataset | Requires the larger volume of dataset compared to machine learning |
| Better for the low-label task. | Better for complex task like image processing, natural language processing, etc. |
| Takes less time to train the model. | Takes more time to train the model. |
| A model is created by relevant features which are manually extracted from images to detect an object in the image. | Relevant features are automatically extracted from images. It is an end-to-end learning process. |
| Less complex and easy to interpret the result. | More complex, it works like the black box interpretations of the result are not easy. |
| It can work on the CPU or requires less computing power as compared to deep learning. | It requires a high-performance computer with GPU. |

**Types of neural networks**

Deep Learning models are able to automatically learn features from the data, which makes them well-suited for tasks such as image recognition, speech recognition, and natural language processing. The most widely used architectures in deep learning are feedforward neural networks, convolutional neural networks (CNNs), and recurrent neural networks (RNNs).

Feed forward neural networks (FNNs) are the simplest type of ANN, with a linear flow of information through the network. FNNs have been widely used for tasks such as image classification, speech recognition, and natural language processing.

Convolutional Neural Networks (CNNs) are specifically for image and video recognition tasks. CNNs are able to automatically learn features from the images, which makes them well-suited for tasks such as image classification, object detection, and image segmentation.

Recurrent Neural Networks (RNNs) are a type of neural network that is able to process sequential data, such as time series and natural language. RNNs are able to maintain an internal state that captures information about the previous inputs, which makes them well-suited for tasks such as speech recognition, natural language processing, and language translation.

**Applications of Deep Learning :**

The main applications of deep learning can be divided into computer vision, natural language processing (NLP), and reinforcement learning.

Computer vision

In computer vision, Deep learning models can enable machines to identify and understand visual data. Some of the main applications of deep learning in computer vision include:

- **Object detection and recognition:** Deep learning model can be used to identify and locate objects within images and videos, making it possible for machines to perform tasks such as self-driving cars, surveillance, and robotics.
- **Image classification:** Deep learning models can be used to classify images into categories such as animals, plants, and buildings. This is used in applications such as medical imaging, quality control, and image retrieval.
- **Image segmentation:** Deep learning models can be used for image segmentation into different regions, making it possible to identify specific features within images.

*Natural language processing (NLP):*

In NLP, the Deep learning model can enable machines to understand and generate human language. Some of the main applications of deep learning in NLP include:

- **Automatic Text Generation** – Deep learning model can learn the corpus of text and new text like summaries, essays can be automatically generated using these trained models.

- **Language translation:** Deep learning models can translate text from one language to another, making it possible to communicate with people from different linguistic backgrounds.
- **Sentiment analysis:** Deep learning models can analyze the sentiment of a piece of text, making it possible to determine whether the text is positive, negative, or neutral. This is used in applications such as customer service, social media monitoring, and political analysis.
- **Speech recognition:** Deep learning models can recognize and transcribe spoken words, making it possible to perform tasks such as speech-to-text conversion, voice search, and voice-controlled devices.

*Reinforcement learning:*

In reinforcement learning, deep learning works as training agents to take action in an environment to maximize a reward. Some of the main applications of deep learning in reinforcement learning include:

- **Game playing:** Deep reinforcement learning models have been able to beat human experts at games such as Go, Chess, and Atari.
- **Robotics:** Deep reinforcement learning models can be used to train robots to perform complex tasks such as grasping objects, navigation, and manipulation.
- **Control systems:** Deep reinforcement learning models can be used to control complex systems such as power grids, traffic management, and supply chain optimization.

**Challenges in Deep Learning**

Deep learning has made significant advancements in various fields, but there are still some challenges that need to be addressed. Here are some of the main challenges in deep learning:

1. Data availability: It requires large amounts of data to learn from. For using deep learning it's a big concern to gather as much data for training.
2. Computational Resources: For training the deep learning model, it is computationally expensive because it requires specialized hardware like GPUs and TPUs.
3. Time-consuming: While working on sequential data depending on the computational resource it can take very large even in days or months.
4. Interpretability: Deep learning models are complex, it works like a black box. it is very difficult to interpret the result.
5. Overfitting: when the model is trained again and again, it becomes too specialized for the training data, leading to overfitting and poor performance on new data.

**Advantages of Deep Learning:**

1. High accuracy: Deep Learning algorithms can achieve state-of-the-art performance in various tasks, such as image recognition and natural language processing.
2. Automated feature engineering: Deep Learning algorithms can automatically discover and learn relevant features from data without the need for manual feature engineering.
3. Scalability: Deep Learning models can scale to handle large and complex datasets, and can learn from massive amounts of data.

4. Flexibility: Deep Learning models can be applied to a wide range of tasks and can handle various types of data, such as images, text, and speech.
5. Continual improvement: Deep Learning models can continually improve their performance as more data becomes available.

**Disadvantages of Deep Learning:**

1. High computational requirements: Deep Learning models require large amounts of data and computational resources to train and optimize.
2. Requires large amounts of labeled data: Deep Learning models often require a large amount of labeled data for training, which can be expensive and time- consuming to acquire.
3. Interpretability: Deep Learning models can be challenging to interpret, making it difficult to understand how they make decisions.
4. Overfitting: Deep Learning models can sometimes overfit to the training data, resulting in poor performance on new and unseen data.
5. Black-box nature: Deep Learning models are often treated as black boxes, making it difficult to understand how they work and how they arrived at their predictions. In summary, while Deep Learning offers many advantages, including high accuracy and scalability, it also has some disadvantages, such as high computational requirements, the need for large amounts of labeled data, and interpretability challenges. These limitations need to be carefully considered when deciding whether to use Deep Learning for a specific task.